

1-Self:

To reduce the JIT compilation overhead at runtime, modern .NET applications use advanced techniques. One key solution is Tiered Compilation, which first runs a quickly compiled, less optimized version of the code to speed up startup time. Then, it fully optimizes the frequently used methods in the background. Another solution is Ahead-of-Time (AOT) Compilation, which compiles the code entirely into native machine code before execution, eliminating the need for JIT at runtime and drastically improving the application's startup performance.

2-linkedIn Article :

Have you ever wondered how code compilation evolved in programming?

Before the .NET Framework (pre-2002), programming faced major challenges. Applications were entirely dependent on the operating system and compiled directly into native machine code. This made it very difficult to mix different languages in one project and restricted the code to a single platform.

But with the launch of the .NET Framework, the game changed! Microsoft introduced a brilliant solution that separated writing code from running it through two main steps: 1-The code (whether written in C# or VB) is first compiled into a unified Intermediate Language (IL/MSIL). This solved the multi-language issue. 2-The Common Language Runtime (CLR) takes over. Using the JIT (Just-In-Time) compiler, it converts this intermediate language into native machine code at runtime.

This evolution didn't just make writing complex applications easier; it laid the foundation for the advanced programming technologies we use today.

3-Report:

1-NET Versions: The .NET platform has evolved significantly over the years. It started with the original .NET Framework, which was limited to Windows. Later, Microsoft introduced .NET Core for cross-platform development. In 2020, they unified the ecosystem into a single platform, starting with ".NET 5" (skipping version 4 to avoid confusion with the old framework). Currently, new versions are released every November, alternating between standard releases and Long-Term Support (LTS) releases.

2-NET Core: Introduced as a major rebuild of the framework, .NET Core was designed with three main goals: to be completely open-source, cross-platform (running seamlessly on Windows, Linux, and macOS), and highly modular. Instead of forcing developers to download the entire framework, it uses a component-based approach via NuGet packages, which makes applications faster and much lighter.

3-Solution: In the .NET ecosystem, a Solution is the highest-level container used to organize code. It acts as a workspace that holds one or more related projects together.

This structure allows developers to easily manage dependencies, configure settings, and build multiple connected projects at the same time.

4-Namespace: A namespace is a method used for the logical grouping and organization of code elements, such as classes and interfaces. Its primary purpose is to prevent naming conflicts in large applications. If you have two classes with the exact same name, placing them in different namespaces keeps the code organized and error-free. Developers can easily access the contents of a namespace using the `using` keyword at the top of their files.